

LALR(1) Automaton and Conflict Analysis Report

1 & 2. LALR(1) Item Sets and Automaton Construction

The LALR(1) automaton successfully constructs **132 total states**. Below is a summary of the automaton metrics and representative initial item sets to demonstrate the construction.

Automaton Summary

- **Total States:** 132
- **Accept States:** 1
- **Conflict States:** 2
- **Terminals:** 29
- **Non-terminals:** 19
- **Productions:** 58

Representative Item Sets

State 0 (Initial)

```
[0] $accept -> . program $end
[1] program -> .
[2] program -> . program statement

Actions: reduce by program -> \epsilon
Goto: program -> State 1
```

State 1

```
[0] $accept -> program . $end
[2] program -> program . statement
[3] statement -> . grid_decl
[4] statement -> . blueprint_decl
[5] statement -> . construct_decl
[6] statement -> . ';'

Actions:
- $end -> State 2 (accept)
- GRID -> 3, BLUEPRINT -> 4, CONSTRUCT -> 5, ';' -> 6

Goto: statement -> 7, grid_decl -> 8, blueprint_decl -> 9, construct_decl -> 10
```

State 2 (Accept)

```
[0] $accept -> program $end .

Action: Accept
```

3 & 4. Conflict Identification and Analysis

The automaton contains **10 total conflicts** confined to **2 states**.

A. Shift/Reduce Conflict: The "Dangling Else"

- **State:** 102
- **Conflict Type:** Shift/Reduce (1 instance)
- **Productions Involved:**
 - [31] `conditional_stmt -> IF '(' condition ')' stmt .`
 - [32] `conditional_stmt -> IF '(' condition ')' stmt . ELSE stmt`

Analysis:

This is the classic dangling-else ambiguity. On the lookahead token `ELSE`, the parser must choose between shifting (attaching the `ELSE` to the current `IF`) or reducing (completing the `IF` statement without an `ELSE`).

Resolution: Bison defaults to `SHIFT`, associating the `ELSE` with the nearest preceding `IF`.

B. Reduce/Reduce Conflict: Block Ambiguity

- **State:** 107
- **Conflict Type:** Reduce/Reduce (9 instances across different lookaheads)
- **Productions Involved:**
 - [26] `stmt_block -> '{' stmt_list '}' .`
 - [33] `repetition_stmt -> REPEAT expr '{' stmt_list '}' .`

Analysis:

Both productions end with the identical suffix `{ stmt_list }`. By the time the parser completes the block, it has lost the context of `REPEAT expr`. Because both reductions are valid on 9 different lookahead tokens (such as `$default`, `ELSE`, `IF`, `REPEAT`, `}`, etc.), the parser cannot determine whether it is closing a standalone block or a repetition statement.

Resolution: Bison resolves this by always selecting the first defined rule (Production 26), effectively making Production 33 unreachable and leading to incorrect parsing of repetition constructs.

Root Causes & Required Grammar Fixes

To resolve the conflicts above, the grammar must be restructured to address the following errors:

1. **Dangling-else:** Can be resolved via precedence declarations or explicit matched/unmatched statement rules.
2. **Standalone block overlaps:** The rules `stmt -> stmt_block` and `repetition_stmt -> REPEAT expr '{' stmt_list '}'` create suffix overlap.
3. **Extended repetition forms:** Multiple repetition rules (`REPEAT expr '{' stmt_list '}'`, `REPEAT '{' stmt_list '}'`, `REPEAT expr stmt`) complicate block resolution.
4. **Missing unary precedence:** Absence of precedence for unary minus (noted in the grammar review).